

Algoritmos de Ordenação: Heapsort

Centro de Informática
Universidade Federal de Pernambuco
Sistemas de Informação

Vinicius Cardoso Garcia
vcg@cin.ufpe.br

Referências

- Notas de aula do prof. Tiago A. E. Ferreira (UFRPE)
- How to Think Like a Computer Scientist
 - <http://www.greenteapress.com/thinkpython/thinkCSpy>
- Aprenda Computação com Python
 - <http://www.franciscosouza.com.br/aprendacompy/>

Introdução

- Problema
 - Dados N números, arranjá-los em ordem crescente ou decrescente
- Ordenação (Sorting)
 - Ordenar é dispor os elementos de um conjunto em uma ordem ascendente ou decendente
- Motivação
 - Otimização de alguns processos
 - Ex.: busca

Exemplo



Premissas

- Humanos fazem ordenação de forma eficiente em muitos casos
- Humanos podem ver todos os jogadores de uma vez e rapidamente rearranjar os elementos de forma *ad hoc*
- Computadores conseguem apenas comparar dois elementos por vez

Algoritmos de Ordenação

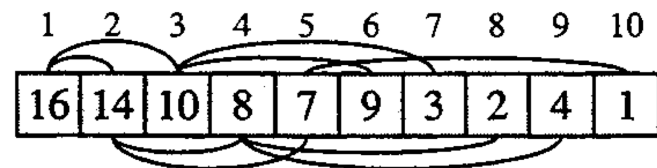
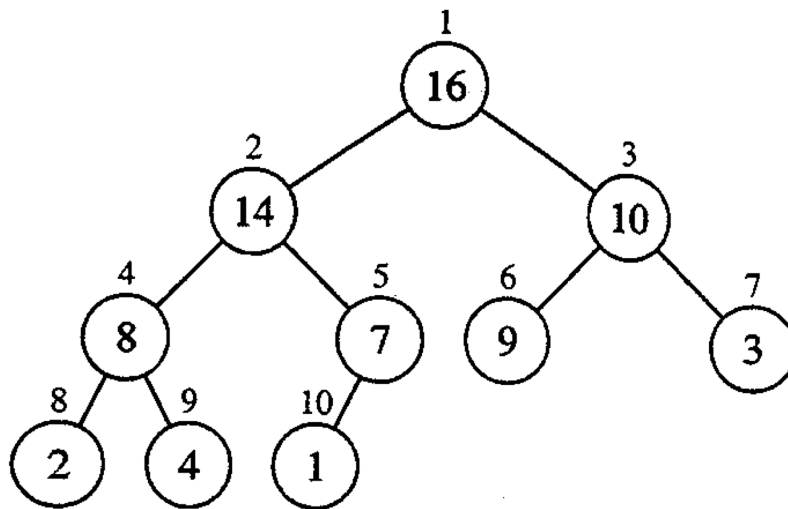
- Os algoritmos de ordenação são aplicados para a organização de listas de elementos quaisquer em ordem crescente ou decrescente
- Existem vários algoritmos para este fim:
 - Bubble Sort
 - Insert Sort
 - Merge Sort
 - Heap Sort
 - Quick Sort
 - Etc...

Algoritmo Heap Sort

- O algoritmo ***Heap Sort*** utiliza o melhor dos algoritmos de ordenamento já estudados até agora:
 - ***Insertion Sort***:
 - Utiliza um ordenamento local
 - ***Merge Sort***:
 - Apenas um número constante de elementos é mantido fora do arranjo de entrada para ser ordenado (dividir para conquistar)

Algoritmo Heap Sort

- O algoritmo **Heap Sort** utiliza uma estrutura de dados chamada **Heap**.
 - Um **Heap** é um objeto arranjo que pode ser visto como uma árvore binária incompleta
 - Cada Nó corresponde a um elemento do arranjo



Estrutura Heap

- O Arranjo representa o *Heap*
 - Este tem dois atributos:
 - **Comprimento[A]**: número de elementos do arranjo A
 - **Tamanho-do-Heap[A]**: número de elementos do Heap arranjado dentro do arranjo A.
 - O arranjo **A[1..comprimento[A]]** pode ter vários elementos
 - Porém, apenas os elementos até o índice **tamanho-do-heap[A]** ($\leq$ **comprimento[A]**) farão parte do *heap*

Estrutura Heap

- O primeiro elemento do arranjo é o nó raiz
- Para um nó i , é possível verificar:
 - O nó Pai: $\text{PARENT}(i)$
 $\text{return } \lfloor i/2 \rfloor$
 - O nó Filho Esquerdo: $\text{LEFT}(i)$
 $\text{return } 2i$
 - O nó Filho Direito: $\text{RIGHT}(i)$
 $\text{return } 2i + 1$
- Estas três operações, na grande maioria dos processadores podem ser executadas em uma única instrução (custo em **tempo constante**)

Estrutura Heap

- Existem dois tipos de heap binários:
 - **Heap máximo**
 - **Propriedade:** para todo elemento diferente da raiz tem-se:
 - $A[\text{Parent}[A]] \geq A[i]$
 - Ou seja, o valor de um nó é no **máximo** o valor do seu pai (o maior elemento está na raiz)
 - **Heap mínimo**
 - **Propriedade:** para todo elemento diferente da raiz tem-se:
 - $A[\text{Parent}[A]] \leq A[i]$
 - Ou seja, o valor de um nó é no **mínimo** o valor do seu pai (o menor elemento está na raiz)

Estrutura Heap

- Visualizando um Heap como uma árvore, é possível definir:
 - **Altura de um nó:** número de arestas no caminho descendente simples mais longo do nó até uma folha
 - **Altura do heap:** é a altura do nó raiz
 - **Altura máxima:** para um heap binário de n elementos, a altura máxima será $\Theta(\lg n)$
 - **Custo em tempo:** de forma geral, as operações com o heap terão custo em tempo proporcionais a sua altura, logo $T(n) = \Theta(\lg n)$

Função MAX-HEAPIFY

- Executado em $\Theta(\lg n)$, é a função chave para a manutenção da propriedade do heap máximo
- **Recebe:** A – Arranjo; i – índice.
- **Supõe:** árvores com raízes $\text{left}[i]$ e $\text{right}[i]$ são heaps máximos, mas $A[i]$ não necessariamente.
- **Objetivo:** “Flutuar” $A[i]$ para garantir um heap máximo

Função MAX-HEAPIFY

def Parent(i): **return** i/2

def Left(i): **return** 2*i

def Right(i): **return** 2*i+1

def MaxHeapify(A, i, n):

"Funcao q garante a manutencao do heap maximo"

l = Left(i)

r = Right(i)

if l <= n **and** A[l] > A[i]: largest = l

else: largest = i

if r <= n **and** A[r] > A[largest]:

largest = r

if largest != i:

A[i], A[largest] = A[largest], A[i]

MaxHeapify(A, largest, n)

PARENT(*i*)

return $\lfloor i/2 \rfloor$

LEFT(*i*)

return 2*i*

RIGHT(*i*)

return 2*i* + 1

MAX-HEAPIFY(*A*, *i*)

1 *l* ← LEFT(*i*)

2 *r* ← RIGHT(*i*)

3 **if** $l \leq \text{tamanho-do-heap}[A]$ e $A[l] > A[i]$

4 **then** *maior* ← *l*

5 **else** *maior* ← *i*

6 **if** $r \leq \text{tamanho-do-heap}[A]$ e $A[r] > A[\text{maior}]$

7 **then** *maior* ← *r*

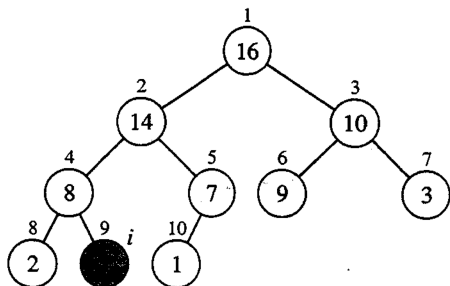
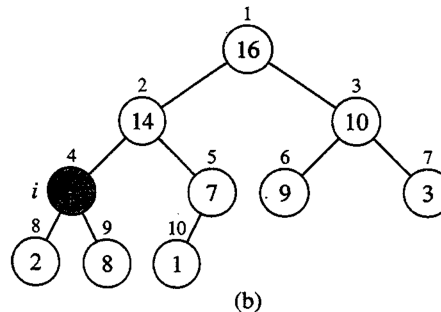
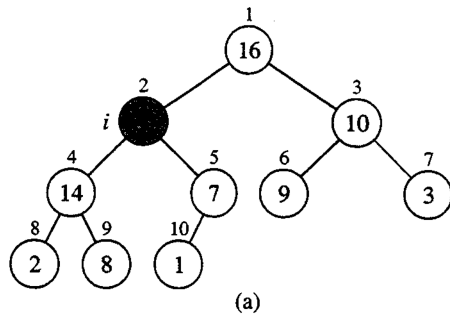
8 **if** *maior* ≠ *i*

9 **then** trocar $A[i] \leftrightarrow A[\text{maior}]$

10 MAX-HEAPIFY(*A*, *maior*)

Função MAX-HEAPIFY

- Suponha o nó $A[2]=4$. Se chamarmos a função $\text{MAX-HEAPIFY}(A,2)$



Função MAX-HEAPIFY

- No pior caso, o custo em tempo irá seguir a recorrência:
 - $T(n) = T(2n/3) + \Theta(1)$
 - Que segundo o teorema mestre, $T(n) = \Theta(\lg n)$

Construindo um Heap Máximo

- É possível utilizar a função MAX-HEAPIFY para construirmos um heap máximo
 - Dado um arranjo, $A[1..n]$ é possível mostrar que $A[(\lfloor n/2 \rfloor + 1) .. n]$ serão todos folhas
 - Assim, todos estes elementos serão heaps de um elemento que poderão ser utilizados para começarmos a construir o heap máximo
- Dada a função BUILD-MAX-HEAP:

BUILD-MAX-HEAP(A)

```
1 tamanho-do-heap[ $A$ ]  $\leftarrow$  comprimento[ $A$ ]  
2 for  $i \leftarrow \lfloor \text{comprimento}[A]/2 \rfloor$  downto 1  
3   do MAX-HEAPIFY( $A, i$ )
```

Exemplo: Construindo um Heap Máximo

BUILD-MAX-HEAP(A)

```

1 tamanho-do-heap[A] ← comprimento[A]
2 for i ← ⌊comprimento[A]/2⌋ downto 1
3   do MAX-HEAPIFY(A, i)
    
```

def HeapLength(A): **return** len(A)-1

def BuildMaxHeap(A)

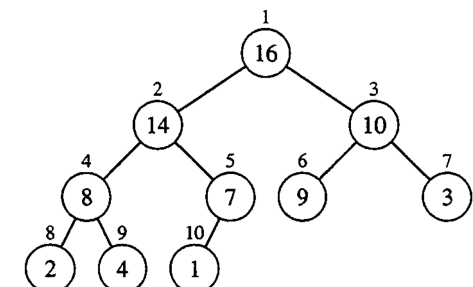
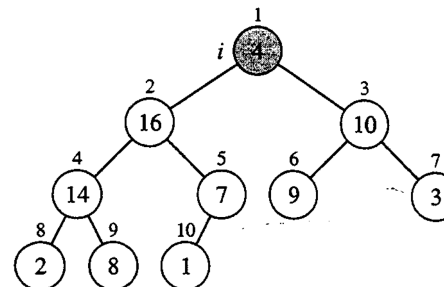
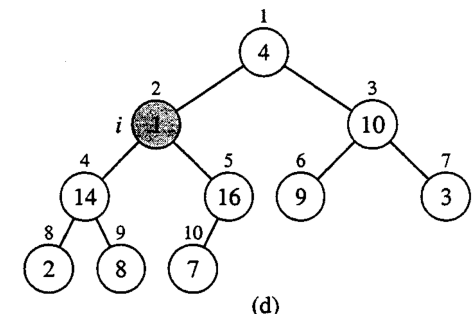
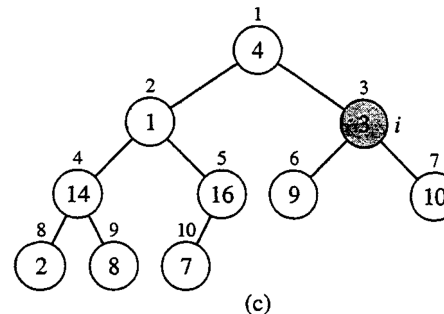
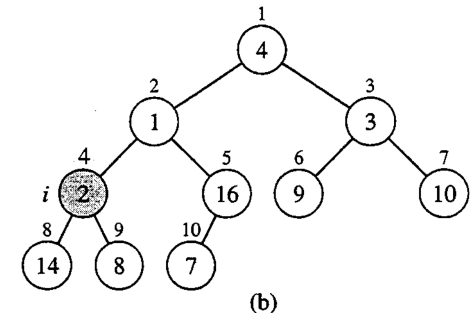
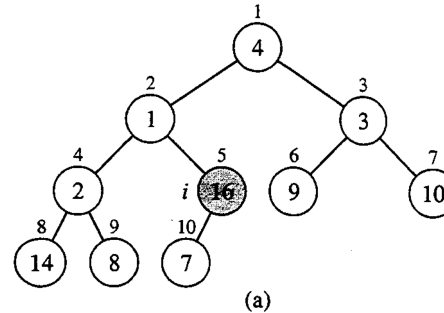
"Funcao que constroi um heap maximo"

n = HeapLength(A)

for i **in** range(n/2,-1,-1):

 MaxHeapify(A,i,n)

A [4 | 1 | 3 | 2 | 16 | 9 | 10 | 14 | 8 | 7]



Custos

- Um heap com **n** elemento terá **altura** $\lfloor \lg n \rfloor$
 - Porém, é possível mostrar que para uma altura **b**, existirá no máximo $\lceil n/2^{b+1} \rceil$ elementos
 - A função MAX-HEAPIFY terá um custo **O(b)** para uma altura **b**
 - Assim, o custo para a função BUILD-MAX-HEAP será:

$$\sum_{b=0}^{\lfloor \lg n \rfloor} \left\lceil \frac{n}{2^{b+1}} \right\rceil O(b) = O\left(n \sum_{b=0}^{\lfloor \lg n \rfloor} \frac{b}{2^b}\right)$$

- Mas $\sum_{b=0}^{\infty} \frac{b}{2^b} = \frac{1/2}{(1-1/2)^2} = 2$ logo $O\left(n \sum_{b=0}^{\lfloor \lg n \rfloor} \frac{b}{2^b}\right) = O\left(n \sum_{b=0}^{\infty} \frac{b}{2^b}\right) = O(n)$
- Assim é possível construir um heap máximo em **tempo linear**!

Algoritmo HeapSort

- Dado um arranjo $A[1..n]$ é criado um heap máximo.
 - Como o maior elemento está da raiz ($A[1]$), troca-se este elemento como o último elemento do arranjo ($A[n]$), diminuindo-se o tamanho do heap de 1.
 - O com o novo arranjo $A[1..(n-1)]$ refaz-se o mesmo procedimento.

HEAPSORT(A)

1 BUILD-MAX-HEAP(A)

2 **for** $i \leftarrow \text{comprimento}[A]$ **downto** 2

3 **do** trocar $A[1] \leftrightarrow A[i]$

4 $\text{tamanho-do-heap}[A] \leftarrow \text{tamanho-do-heap}[A] - 1$

5 MAX-HEAPIFY($A, 1$)

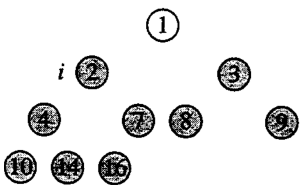
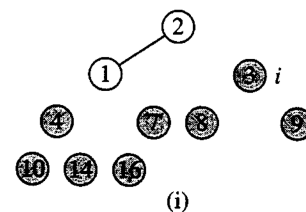
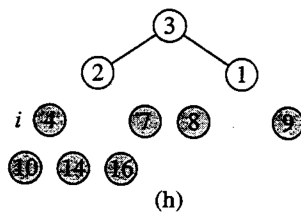
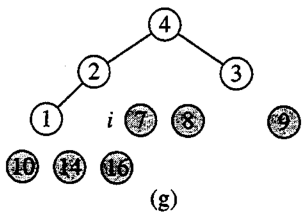
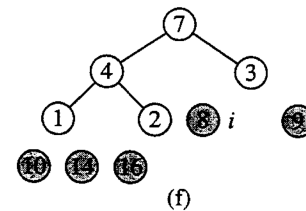
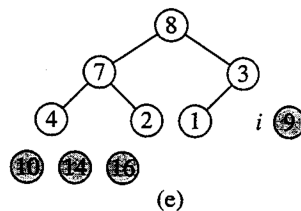
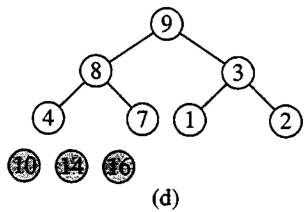
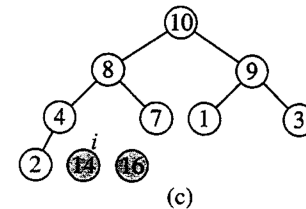
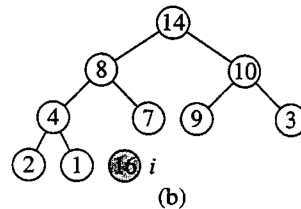
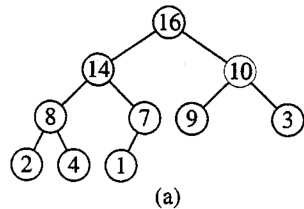
Algoritmo HeapSort (Python)

```
def HeapSort(A): # use a heap to build sorted array from the end
    "Algoritmo de ordenamento HeapSort"
    BuildMaxHeap(A)
    HeapSize=HeapLength(A)
    for i in range(HeapSize,0,-1):
        A[0],A[i]=A[i],A[0] # largest element is a root of heap, put it at the end of array
        HeapSize=HeapSize-1 # shrink heap size by 1 to get next largest element
        MaxHeapify(A,0,HeapSize)
    return A
```

Custo do HeapSort

- Cada chamada da função BUILD-MAX-HEAP tem custo **$O(n)$** .
 - Esta função é chamada **1** vezes!
- Cada chamada da função MAX-HEAPIFY tem custo **$O(\lg n)$**
 - Esta função é chamada **$(n-1)$** vezes!
- O custo total do algoritmo HeapSort será:
 - **$O(n \lg n)$**

Exemplo



A [1 | 2 | 3 | 4 | 7 | 8 | 9 | 10 | 14 | 16]

Atividades complementares

- Implemente o HeapSort em Python OO
 - E em Java
- Leitura dos Capítulo 6 do livro do Cormen e Resolva as seguintes questões:
 - 6.1-2
 - 6.1-7
 - 6.3-3
- Hints for implementing HeapSort & Priority Queues in Python
 - <https://eee.uci.edu/01f/15545/heap-hint.html>